Ders 9 Çalışma Özeti

Ders 9 Çalışma Özeti

10 Aralık 2020 Perşembe, online ders (yaklaşık 2 saat 14 dakika). Anlatım İngilizce, proje kısmı Türkçe. Konu: Bing Liu, *Web Data Mining*, Chapter 8 (Web Crawling). Slaytlar Filippo Menczer'in (Indiana University) kitap için hazırladığı sunumlar.

Genel Konular

- Proje önerilerine verilen geri bildirimler
- Tarayıcıların (crawler) amacı ve sınıflandırması (taxonomy)
  - Evrensel (universal) ve tercihli (preferential) tarayıcılar
  - Odaklı (focused) ve konulu (topical) tarayıcılar
  - Statik ve uyarlamalı (adaptive) konulu tarayıcılar
- Temel sıralı tarayıcı (basic sequential crawler) ve modülleri
  - Frontier listesi, fetcher, parser, repository
  - Genişlik öncelikli (breadth-first) ve derinlik öncelikli (depth-first) gezinme
  - Perl ile basit tarayıcı
- Gerçekleştirme sorunları (implementation issues)
  - Aynı sayfayı tekrar indirmeme, frontier önceliklendirme
  - Zaman aşımı (timeout), içerik türü kontrolü, boyut sınırı
  - Bozuk HTML ve DOM ağacı
  - Durak kelime, kök bulma, eş anlamlı kelimeler
  - Göreli/mutlak URL, kanonik URL, örümcek tuzakları (spider traps)
  - Sayfaların saklanması
- Eşzamanlı (concurrent) çok iş parçacıklı tarayıcı
- Evrensel tarayıcılar: kapsam ile tazelik (coverage vs freshness) dengesi
- Tercihli tarama algoritmaları: Best-N-First, PageRank, SharkSearch, InfoSpiders
- Tarayıcı değerlendirmesi: tarama boyutu ve recall grafiği
- Açık kaynak tarayıcı kütüphaneleri
- Proje teslimi ve demo kuralları

Hocanın Özellikle Vurguladığı Kısımlar

- Bu bölüm önemli, çünkü projelerin çoğu bir web tarayıcısı geliştiriyor.
- Frontier listesine konacak tüm URL'ler mutlak (absolute) ve kanonik (canonical) biçimde olmalı.
- Frontier'a eklemeden önce URL'nin daha önce indirilip indirilmediği kontrol edilmeli.
- Fetcher sağlam (robust) olmalı ve zaman aşımı mekanizması içermeli.
- Eşzamanlı tarayıcıda frontier paylaşılan veri yapısıdır. Erişim frontier manager üzerinden senkronize edilmeli.
- Projeler için:
  - Basit mimarili bir kütüphane tercih edilmeli (ör. Java'da crawler4j).
  - JavaScript'i az kullanan, içeriği HTML ile sunan e-ticaret siteleri seçilmeli.
  - Her proje bir kullanıcı arayüzü olan çalışan bir uygulama olmalı. Jupyter Notebook, konsol çıktısı ya da slayt kabul edilmez. Uygulama olmayan proje notlandırılmaz.
- Bağlantı analizi (link analysis) önemli bir konu ve sınavda çıkacak.

Kısa Tekrar Notları

- Tarayıcının diğer adları: spider, robot, web agent. Örnekler: Googlebot, Bingbot.
- Tarayıcı aramadan önce, arka planda sürekli çalışır. HTML sayfalarını merkezi bir depoya indirir, indeks bu depodan oluşturulur.
- Temel döngü:
  1. Tohum URL'lerle (seed URLs) frontier listesi başlatılır.
  2. Listenin başından bir URL alınır ve sayfa indirilir.
  3. Sayfadaki URL'ler çıkarılıp frontier'a eklenir.
  4. Sayfa depoya yazılır.
  5. Durma koşulu sağlanana kadar devam edilir.
- Durma koşulu: belli sayıda sayfa indirmek, belli derinliğe (ör. 3. seviye) ulaşmak ya da frontier'ın boşalması.
- Genişlik öncelikli gezinme FIFO (kuyruk), derinlik öncelikli gezinme LIFO (yığın) ile çalışır.
- Önce HTTP `HEAD` isteğiyle içerik türü (Content-Type) öğrenilir, uygunsa `GET` ile sayfa indirilir.
- Örümcek tuzağına karşı basit kural: 128 karakterden uzun URL'ler frontier'a alınmaz.
- Tazelik için HTTP başlığındaki `Last-Modified` ve `Expires` alanları kullanılır.
- Tercihli tarayıcılar recall ile tarama boyutu (crawl size) grafiğiyle karşılaştırılır. Az sayfa tarayarak çok ilgili sayfa bulan algoritma daha iyidir.

Detaylı Açıklamalar

Proje Geri Bildirimleri (Dersin Başı)

Hoca proje önerileri için hazırladığı notları bir dosyada yayımladı:

- Özellikle "kabul edilmedi" yazılmayan projeler kabul edildi. Bunların önerilerinde yalnızca eksikler var:
  - Hangi veri setinin kullanılacağı belirtilmemiş.
  - Hangi algoritmaların kullanılacağı belirtilmemiş.
  - Yalnızca tek bir algoritmadan söz edilmiş.
- Birkaç satırlık öneriler kabul edilmedi.
- Önerileri tekrar göndermek zorunlu değil. Hoca sunum sırasında bu notların karşılanıp karşılanmadığına bakacak.
- Sınıflandırma projelerinde en az 3 farklı sınıflandırma algoritması bekleniyor. Tarayıcı projelerinde BFS'in yanında DFS ve Best-First algoritmaları da isteniyor.

Tarayıcı Neden Gerekir?

- Arama motorları gerçek zamanlı aramayı ters indeks üzerinde yapar. İndeksi oluşturmak için sayfaların önce indirilmesi gerekir.
- Dikey (vertical) ya da özelleşmiş tarayıcılar belirli sitelerden veri toplar: haber, alışveriş, makale, yemek tarifi, yorum siteleri.
- İş zekâsı (business intelligence): rakiplerin sitelerini izlemek.
- İlgilenilen siteleri değişiklikler için izlemek.
- Kötü amaçlı kullanım: e-posta adresi toplayıp spam ya da oltalama (phishing) için veritabanı oluşturmak.
- Büyük motorlar indekslerini hem büyük hem güncel tutmak için sürekli tarama yapar.

Tarayıcı Sınıflandırması

- Evrensel tarayıcı: genel arama motorlarını destekler, her türlü HTML sayfasını toplar.
- Tercihli tarayıcı: yalnızca belirli bir konudaki sayfaları toplar.
  - Odaklı tarayıcı (focused crawler):
    - Belirli bir konu için indeks oluşturur (ör. elektronik e-ticaret siteleri).
    - İndirilen sayfanın konuya ait olup olmadığına karar vermek için sayfaları sınıflandırabilir.
  - Konulu tarayıcı (topical crawler):
    - İndeks oluşturmaz, arama sırasında web'i tarar.

    * Sayfa konuya yeterince benziyorsa ondan çıkan URL'leri izlemeye devam eder. Benzerlik eşiğin altındaysa o sayfanın bağlantılarını izlemez.
    * Statik konulu tarayıcı önceden tanımlı bir gezinme algoritması kullanır: PageRank, best-first, breadth-first, depth-first.
    * Uyarlamalı konulu tarayıcı taradığı sayfalardan öğrenir. Örneğin "web data mining" ararken bir sayfada "information retrieval" sık geçiyorsa bu terimi de konuya ekler.

Temel Sıralı Tarayıcı

Modüller:

- Initialize frontier: tohum URL'leri alıp frontier listesini oluşturur. Liste genelde bellekte bağlı liste (linked list) olarak tutulur.
- Dequeue: listenin başındaki URL'yi alır.
- Fetch:
  - URL için HTTP isteği oluşturur ve sunucuya gönderir.
  - HTTP yanıt nesnesinden HTML'i çıkarır.
- Parse: HTML'deki URL'leri çıkarır ve frontier'a ekler.
- Store: HTML'i depoya (repository) yazar.
- Durma koşulu sağlanmadıysa Dequeue adımına dönülür.

Gezinme stratejileri:

- Genişlik öncelikli: web grafiğini seviye seviye tarar. Önce eklenen önce alınır (FIFO, kuyruk).
- Derinlik öncelikli: bir dalın tüm düğümlerini bitirip sonra diğer dala geçer (LIFO, yığın).

Slayttaki Perl kodu (genişlik öncelikli, kuyruk `shift` ve `push` ile):

```
my @frontier = read_seeds($file);
while (@frontier && $tot < $max) {
    my $next_link = shift @frontier;
    my $page = fetch($next_link);
    add_to_index($page);
    my @links = extract_links($page, $next_link);
    push @frontier, process(@links);
}
```

- Döngü frontier boşalana ya da indirilen sayfa sayısı (`tot`) üst sınıra (`max`) ulaşana kadar sürer. Üst sınır yapılandırılabilir bir parametredir.
- `extract_links` mevcut sayfanın kendi URL'sini dışarıda bırakır, böylece aynı sayfa tekrar indirilmez.

Gerçekleştirme Sorunları

- **Tekrar indirmeme:** Ziyaret edilen sayfalar bir tabloda tutulur. URL frontier'a eklenmeden önce bu tabloya bakılır.
- **Frontier'ın hızlı büyümesi:**
  - Her sayfadan çok sayıda URL çıkar.
  - Sınırlı zamanda önemli sayfaları taramak için önemli URL'ler listenin başına konur (öncelikli kuyruk).
- **Fetcher:**
  - Ağ gecikmesi yüksek ya da bant genişliği dar sunucularda uzun beklenebilir. Zaman aşımı konur; süre dolunca sonraki URL'ye geçilir.
  - Yalnızca ayrıştırılıp indekslenebilecek türler (HTML, Word, metin) indirilir. Bunun için önce `HEAD` ile içerik türü sorulur, uygunsa `GET` yapılır.
  - Boyut sınırı konabilir: ör. 100 KB'tan büyük dokümanlar indirilmez ya da yalnızca ilk 100 KB'ı indirilir.
- **Ayrıştırma (parsing):**
  - Web'deki HTML'ler çoğu zaman kurallara uygun yazılmaz, eksik etiketler içerir.
  - Tarayıcılar (Chrome, Edge) bu eksiklere rağmen sayfayı gösterebilir; ayrıştırıcı da öyle olmalıdır.

  - Bunun için HTML'den bir DOM ağacı (Document Object Model tree) kurulur ve URL'ler bu ağaçtan çıkarılır.
- **Depolama ve metin işleme:**
  - Sayfalar depoya olduğu gibi yazılabilir ya da içerikleri çıkarılarak yazılabilir.
  - Çoğu tarayıcı sayfayı olduğu gibi saklar. Durak kelime çıkarma, kök bulma, eş anlamlıları birleştirme (conflation) gibi işlemler indeks oluşturulurken uygulanır.
- **Göreli ve mutlak URL:**
  - Aynı alan adındaki bağlantılar göreli (relative) yazılabilir.
  - Göreli URL, sayfanın temel URL'si (base URL) ile birleştirilip mutlak URL'ye çevrilir.
- **Kanonik URL:**
  - Aynı sayfa farklı yazımlarla gelebilir: büyük/küçük harf, gereksiz port numarası, `../` içeren yollar.
  - Hepsi tek bir kanonik biçime indirgenir.
- **Örümcek tuzakları:**
  - Genellikle CGI gibi betiklerle her tıklamada yeni URL üretilen sayfalardır. Amaç tarayıcıyı aynı alt alan adında hapsetmektir.
  - Önlem: URL uzunluğu eşik değeri (ör. 128 karakter) aşarsa URL atılır.
- **Sayfaları saklama:**
  - Her sayfa ayrı dosya olarak saklanabilir; dosya adı URL'nin özetinden (hash) üretilir. Ancak milyonlarca dosyayı yönetmek zordur.
  - Tüm sayfalar tek büyük bir XML dokümanında saklanabilir.
  - SQL Server gibi ilişkisel veritabanlarının yükü fazladır. Berkeley DB gibi hafif veritabanları önerilir.

Eşzamanlı Tarayıcı

Sıralı tarayıcıda zaman alan işler:

- DNS çözümleme: her URL için IP adresi sorulur.
- Senkron soket: istemci yanıt gelene kadar bekler.
- Sayfanın alınması.

Çok iş parçacıklı mimari:

1. Frontier manager tohum URL'leri frontier'a koyar ve tarayıcı iş parçacıklarını başlatır.
2. Her iş parçacığı sıralı tarayıcı gibi çalışır: frontier manager'dan URL ister, sayfayı indirir, ayrıştırır, URL'leri geri ekletir, sayfayı depoya yazar.
3. Yüzlerce iş parçacığı frontier boşalana kadar aynı işi paralel yapar.

Paylaşılan veri yapıları frontier ve depodur:

- Aynı sayfanın iki kez indirilmesi engellenirse depoya yazmada çakışma olmaz.
- Frontier'dan URL alırken birden fazla iş parçacığı yarışır. Bu yüzden senkronizasyon frontier manager'da yapılır.

Asenkron soket yanıtı beklemeden başka bir isteğin işlenmesine izin verir.

Evrensel Tarayıcılar

- Google, Bing, Yahoo, Yandex gibi motorları destekler. Çok büyük ölçeklidirler, indeksi sıfırdan değil artımlı (incremental) olarak oluştururlar.
- Kapsam ile tazelik arasında denge kurulur:
  - Kapsam: verilen sürede olabildiğince çok sayfa taramak, indeksi milyarlarca sayfaya büyütmek.
  - Tazelik: daha küçük bir indeksi sık sık yeniden tarayarak güncel tutmak. Örneğin iki ay önce indirilen bir sayfanın içeriği değişmiş olabilir.
- İndeks boyutunun yıldan yıla değişmesi bu tercihin değişmesiyle açıklanabilir.
- Hızlandırma çözümleri:
  - DNS sonuçları için önbellek (cache).
  - Asenkron soketler.
  - Yazılım tabanlı ağ ve depolama anahtarları.
- Tazelik HTTP yanıt başlığındaki `Last-Modified` ve `Expires` alanlarıyla izlenir. `Expires` tarihi gelince sayfa yeniden taranır.

Tercihli Tarama Algoritmaları

Tercihli tarayıcı frontier'ı, URL'lerin konuyla benzerliğine göre önceliklendirir. Örnek konular: yemek tarifi, haber, televizyon, iPhone.

- Breadth-first: karşılaştırma için temel (benchmark) olarak kullanılır.
- Best-N-First:
  - URL'ler konuya benzerliklerine göre öncelikli kuyruğa konur.
  - Çok iş parçacıklı yapıda en iyi N URL aynı anda taranır.
- PageRank: URL'ler PageRank değerlerine (bağlantı tabanlı popülerlik) göre sıralanır.
- SharkSearch:
  - URL'nin kendi metnini ve bağlantı metnini (anchor text) konuyla karşılaştırır.
  - URL'nin çıktığı üst (parent) sayfanın içeriğini de konuyla karşılaştırır.
  - Bu benzerliklere göre frontier'ı yeniden sıralar.
- InfoSpiders: uyarlamalı konulu tarayıcıdır. İndeks oluşturmaz, benzer sayfalardan yeni anahtar kelimeler öğrenip konuya ekler.

Değerlendirme grafiği x ekseninde tarama boyutunu, y ekseninde recall'u gösterir. Karşılaştırılan stratejiler:

- Rastgele
- Genişlik öncelikli
- Geri bağlantı (backlink) sayısına göre
- PageRank'e göre

Her strateji, tüm sayfalar tarandığında %100 recall'a ulaşır. Hocaya göre taramanın %20'sinde PageRank ilgili sayfaların yaklaşık %60'ını bulmuştu, yani en başarılı strateji oydu. İyi bir tercihli tarayıcı küçük tarama boyutlarında yüksek recall yakalar.

Konulu tarayıcı arayüzünde kullanıcı anahtar kelimeyi girer. Tarama hemen başlar ve bulunan sayfalar benzerlik puanına göre sıralanarak gösterilir.

Açık Kaynak Kütüphaneler

Perl, Java ve C# için tarayıcı kütüphaneleri var. Hepsi eşzamanlıdır ve anlatılan sorunları çözer. Hoca şunları önerdi:

- Java ile yazılmış Apache Nutch.
- Proje için daha basit mimarili crawler4j. Böylece öğrenciler ayrıntılarda kaybolmadan kendi modüllerini üstüne kurabilir.

Proje Teslimi ve Demo

Tarayıcı projeleri:

- Birden fazla e-ticaret sitesinden belirli bir ürün grubunun (ör. tüm iPhone ürünleri) verileri çekilmeli.
- Çekme sıklığı (her 5 dakika, her gün vb.) yapılandırılabilir olmalı.
- Her ürün bir tablo satırı gibi gösterilmeli: fiyat, adet, özellikler ve ürünün geldiği sayfanın bağlantısı.
- Arayüzde fiyat, adet ve özelliklere göre sıralama ve filtreleme olmalı.
- Gerekenler:
  - Bir tarayıcı (ya da bir kütüphanenin üstüne yazılmış tarayıcı modülü).
  - E-ticaret sayfalarından ilgili içeriği çıkaran bir ayrıştırıcı.
- Tarama sitenin arama arayüzü elle kullanılarak yapılmamalı. Ana sayfa URL'sinden başlayıp alt sayfalar taranmalı ve ürünler kendi indeksinizde toplanmalı.
- Site bir REST API sunuyorsa kullanılabilir, ama hocanın tercihi tarayıcıdır:
  - Çoğu site böyle bir arayüz sunmaz.
  - Yakalanan API'ler başka IP'den gelen istekleri reddedebilir.
- Aşağı kaydırdıkça yüklenen (sonsuz kaydırma) sayfalar tarayıcılar için sorundur. Nutch'un yeni sürümlerinde buna destek olabilir. Hoca bunun yerine JavaScript'i az kullanan siteleri seçmeyi önerdi.

Veri seti kullanan projeler:

- Demo Jupyter Notebook'ta, konsolda ya da slaytla yapılamaz; bir kullanıcı arayüzü gerekir.
- Sınıflandırma projelerinde arayüzde şunlar seçilebilmeli:
  - Veri seti
  - Algoritma
  - Eğitim/test oranı (%70/%30 gibi)
- Eğitim başlatılınca modeller oluşmalı ve test verisindeki sonuçlar gösterilmeli: accuracy, F1, precision, recall, karışıklık matrisi.
- Kümeleme projelerinde algoritma ve parametre seçilmeli (ör. k-means için k), sonuçlar görselleştirilmeli.

Teslim ve demo:

- Teslim dönemin son haftasında.
- Demolar son haftanın Perşembe günü yapılacak. Ders saat 9'da başlayıp 13'e kadar sürebilir.
- Her grup 15–20 dakika içinde ekran paylaşıp demosunu gösterecek ve nasıl yaptığını anlatacak.
- Hoca editörü açtırıp koda dair sorular sorabilir.
- Gelecek hafta bağlantı analizi (link analysis) işlenecek.

Ders sonrası notlar

- Hoca Microsoft'un tarayıcısını "MSNbot" olarak andı. MSNbot, 2010'dan beri Bing'in tarayıcısı Bingbot ile değiştirildi.
- "İnternet bant genişliğinin %60'ını tarayıcılar tüketiyor" ifadesi doğrulanamayan bir tahmindir. Bot trafiğine dair yayımlanan ölçümler genellikle web trafiğinin yaklaşık %40'ını tüm botlara (iyi ve kötü) bağlar; bu payın yalnızca bir kısmı arama motoru tarayıcılarıdır.
- `Last-Modified` ve `Expires` HTML dokümanının değil, HTTP yanıt başlığının alanlarıdır. Hoca bir yerde "HTML dokümanında" dedi.
- Altyazıda "hat" diye geçen komut HTTP `HEAD` yöntemidir.
- Tarama algoritması "Best-N-First"; altyazıda "best and first" olarak geçiyor.
- "Notch" ve "Natch" diye geçen kütüphane Apache Nutch'tır.
- Kanonik URL ve örümcek tuzağı kuralları sezgisel (heuristic) kurallardır. 128 karakter sınırı slayttaki örnek eşiktir, standart bir değer değildir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.